



Module 4 — Configuration des pods

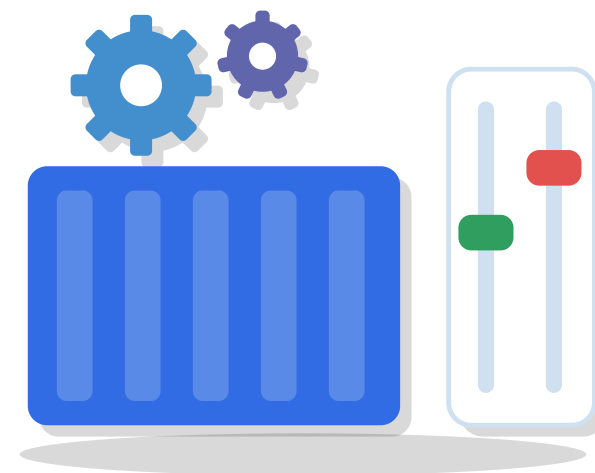
Ressources, stockage, configuration et placement

Kubernetes — Orchestration des conteneurs

Objectifs du module

À la fin de ce module, vous saurez :

- lire et écrire un manifeste de Pod complet, avec init container et sidecar ;
- dimensionner CPU et mémoire avec les **requests** et les **limits**, et prévoir la classe de QoS ;
- donner à TrackR un stockage qui survit aux pods ;
- injecter configuration et secrets ;
- **placer automatiquement** les conteneurs sur les bons nœuds.



Plan du module

1. Anatomie d'un Pod et pods multi-conteneurs
 2. Mémoire et CPU : requests, limits, QoS, OOMKilled
 3. Stockage : volumes, PV, PVC, StorageClass
 4. Configuration applicative : ConfigMap et Secret
 5. Affectation des pods aux nœuds : sélecteurs, affinités, taints, topologie
- Puis la démo 4, le TP 4 (configurer TrackR) et l'exercice QoS.

Toutes les valeurs chiffrées de ce module ont été mesurées sur le lab : kind v1.35, 3 nœuds.

1. Anatomie d'un Pod

Les quatre parties d'un manifeste

```
apiVersion: v1
kind: Pod
metadata:
  name: trackr-api
  labels: {app: trackr-api}
spec:
  containers:
  - name: api
    image: podinfo:6.9.2
    resources: {...}
    env, volumeMounts
  volumes: [...]
  affinity, tolerations
status: ...
```

Type d'objet
et version d'API

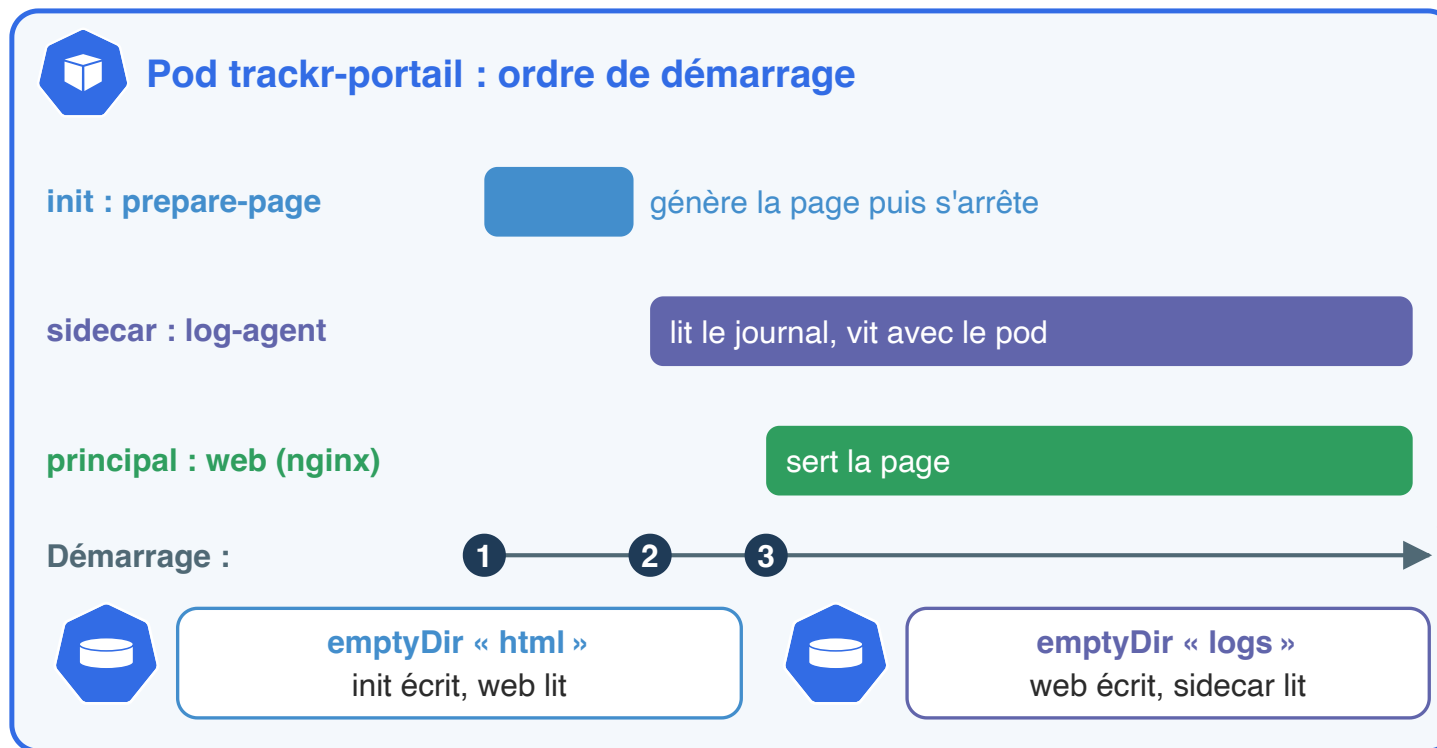
Identité : nom,
labels, annotations

État désiré : conteneurs,
ressources, volumes,
placement

État observé, écrit
par Kubernetes

Vous écrivez `apiVersion`, `kind`, `metadata` et `spec` ; Kubernetes écrit `status`.

Pods multi-conteneurs

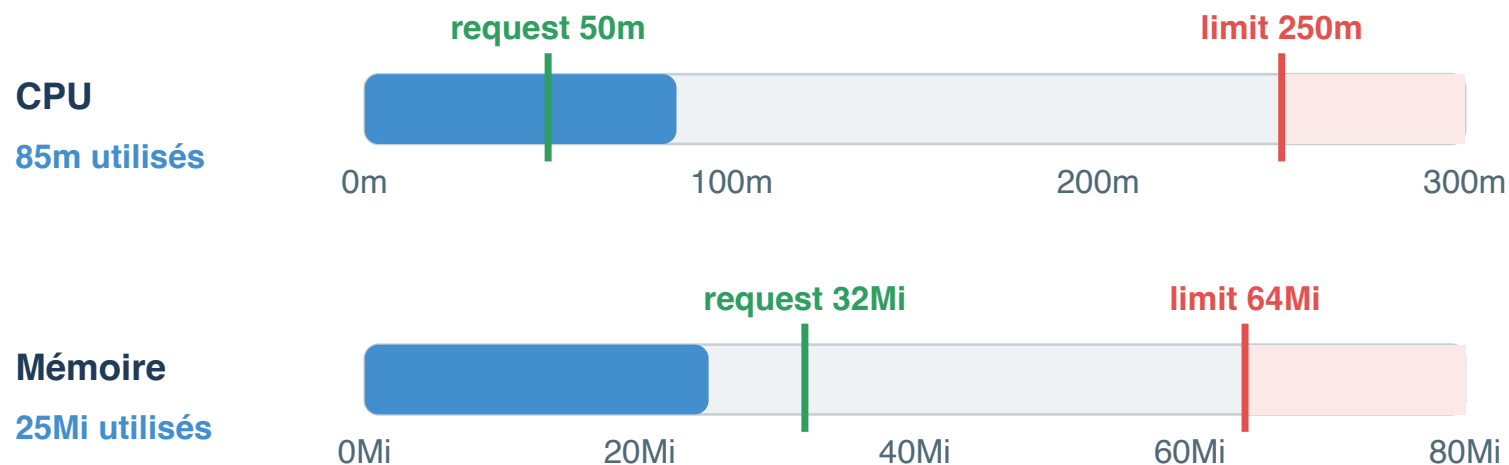


Init : va jusqu'au bout avant les autres. **Sidecar** : `restartPolicy: Always`, vit avec le pod.

2. Mémoire et CPU

Requests et limits

trackr-api sous charge (valeurs mesurées par kubectl top)

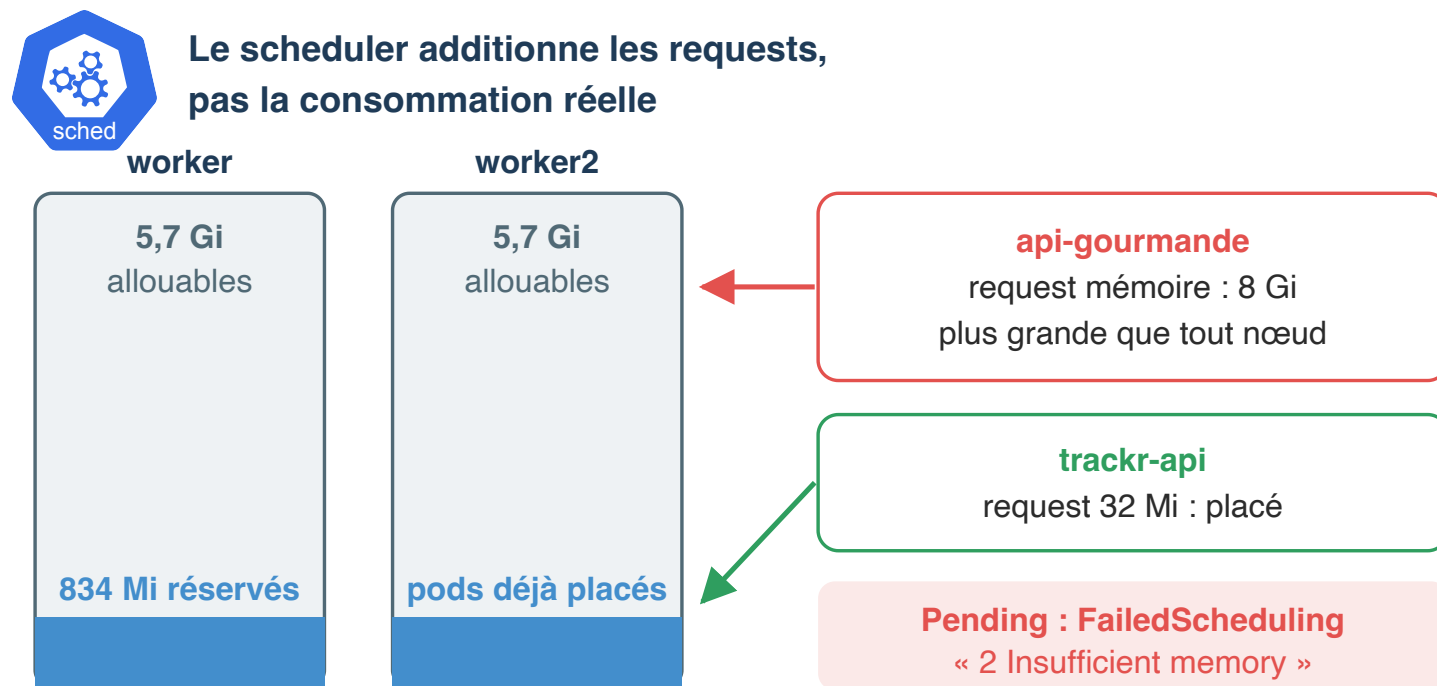


Request : réservée au placement
garantie au conteneur

Limit : plafond à l'exécution
CPU freiné, mémoire : OOMKilled

Unités : 250m = 0,25 cœur ; mémoire en Mi / Gi. 64m (0,064 octet) bloque le pod.

Les requests décident du placement

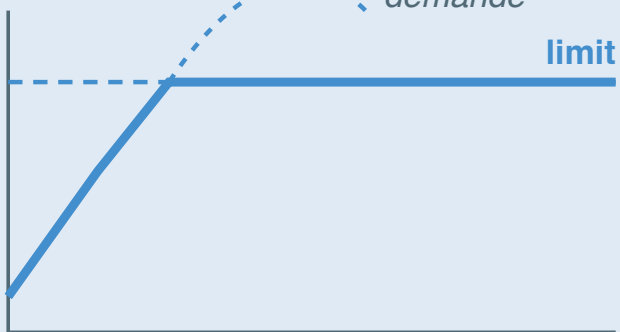


Un pod qui demande plus que l'allouable d'un nœud reste **Pending** à vie.

CPU freiné, mémoire tuée

CPU : compressible

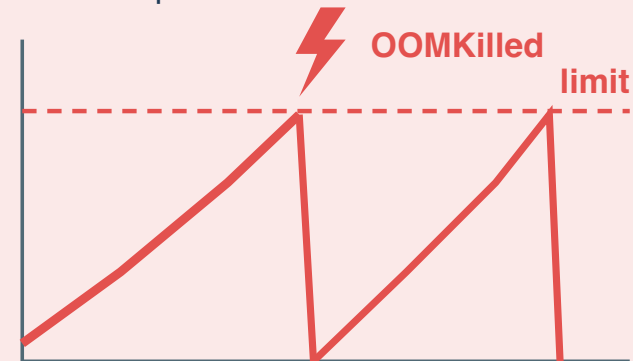
dépassement = freinage
demande



Le conteneur continue, plus lentement
Mesuré : 200m consommés
pour une limite de 200m

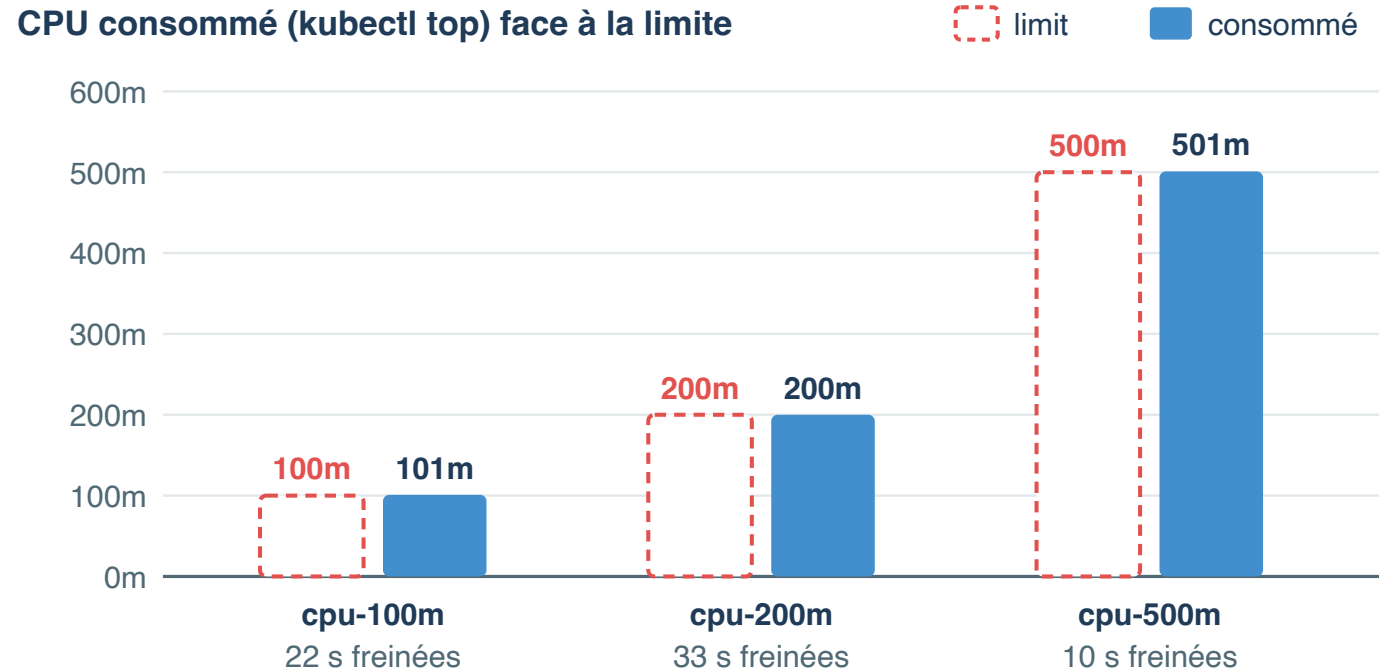
Mémoire : incompressible

dépassement = arrêt brutal



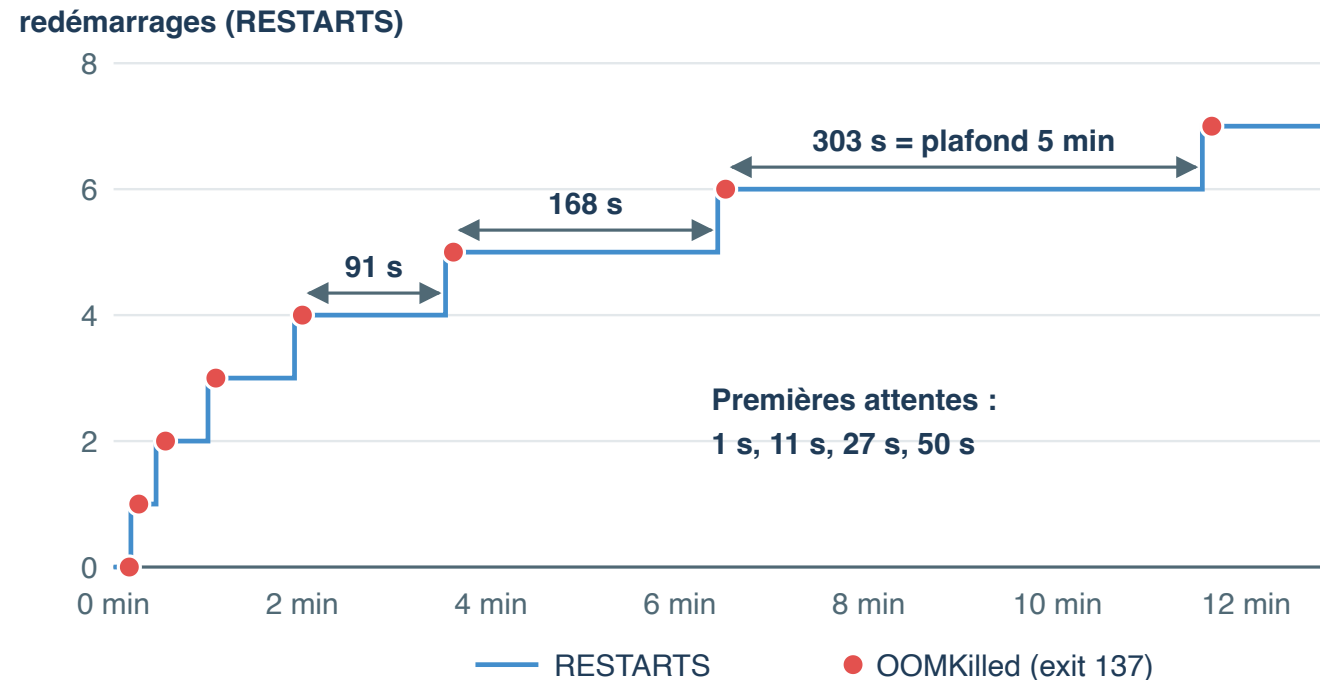
Tué par le noyau (exit 137)
Mesuré : 8 OOMKilled en 12 min,
attente jusqu'à 5 min

Mesure : le throttling CPU



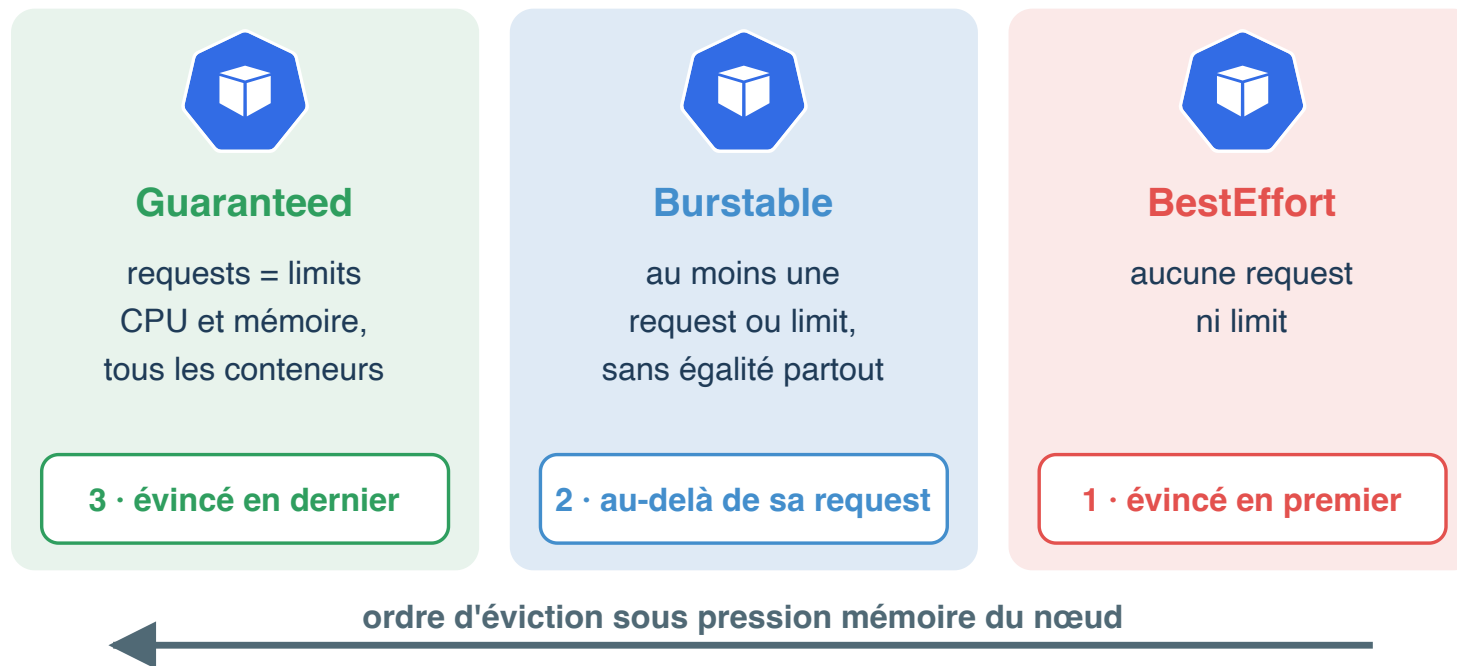
Boucle infinie, trois limites : la consommation colle à la limite, sans aucun redémarrage.
Mesuré sur le lab, kind v1.35, 3 nœuds.

Mesure : OOMKilled et backoff



Limite 100 Mi, allocation de 250 Mo : 8 OOMKilled en 12 min, attente doublée jusqu'à 5 min (CrashLoopBackOff). Mesuré sur le lab.

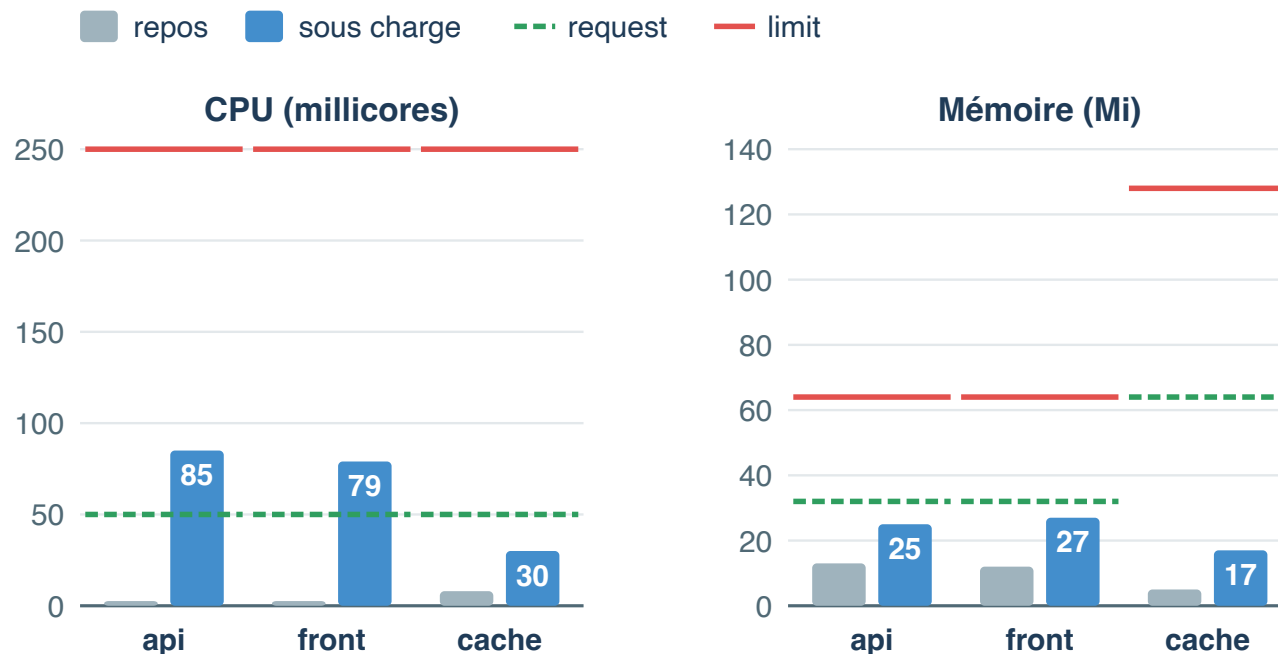
Classes de QoS et éviction



TrackR (requests < limits) : tous les pods sont Burstable

La classe se lit dans `kubectl get pod -o jsonpath='{.status.qosClass}'`.

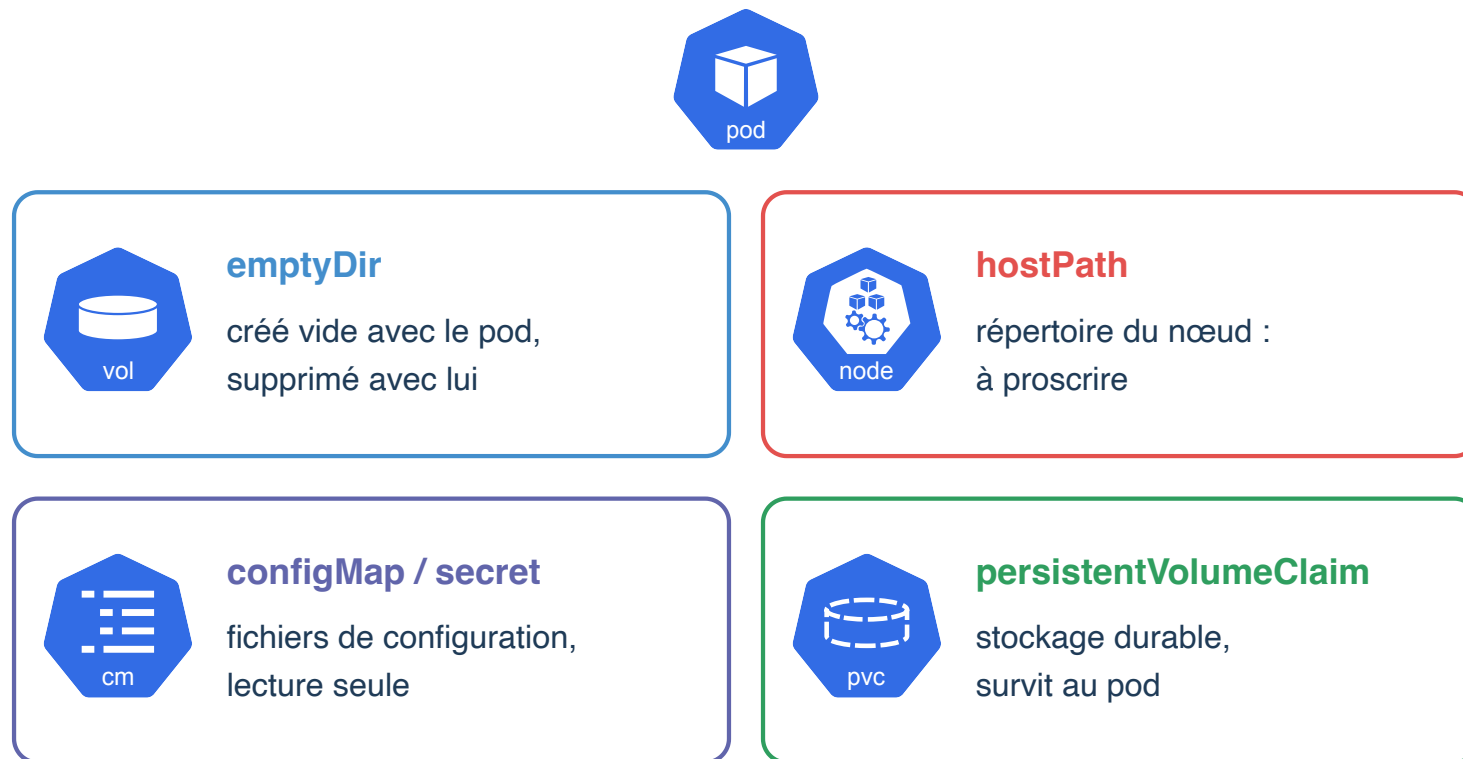
Mesure : TrackR avec kubectl top



Au repos 1m et 12 Mi par pod podinfo ; sous charge 85m et 25 Mi : les limits laissent une marge suffisante. Mesuré sur le lab.

3. Stockage

Les volumes d'un Pod



Un volume est déclaré dans `spec.volumes`, puis monté par chaque conteneur via `volumeMounts`.

hostPath : le danger

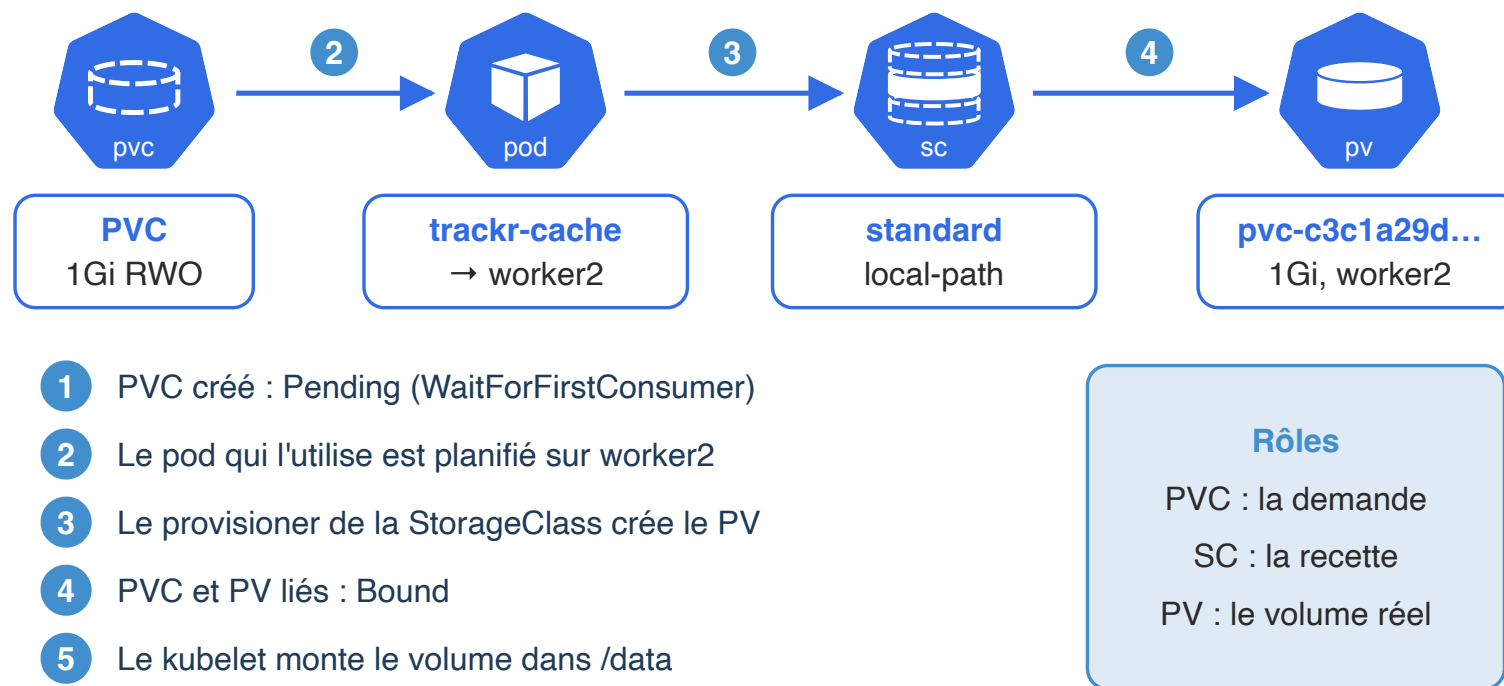
Un pod monté sur `/` du nœud, **en lecture seule**, a pu lire sur le lab :

- la clé privée du kubelet, `kubelet.key` ;
- les données Redis de TrackR dans le répertoire du PV.

Réservez hostPath aux agents d'infrastructure. Pod Security Admission « baseline » l'interdit (module 7).



PV, PVC et StorageClass



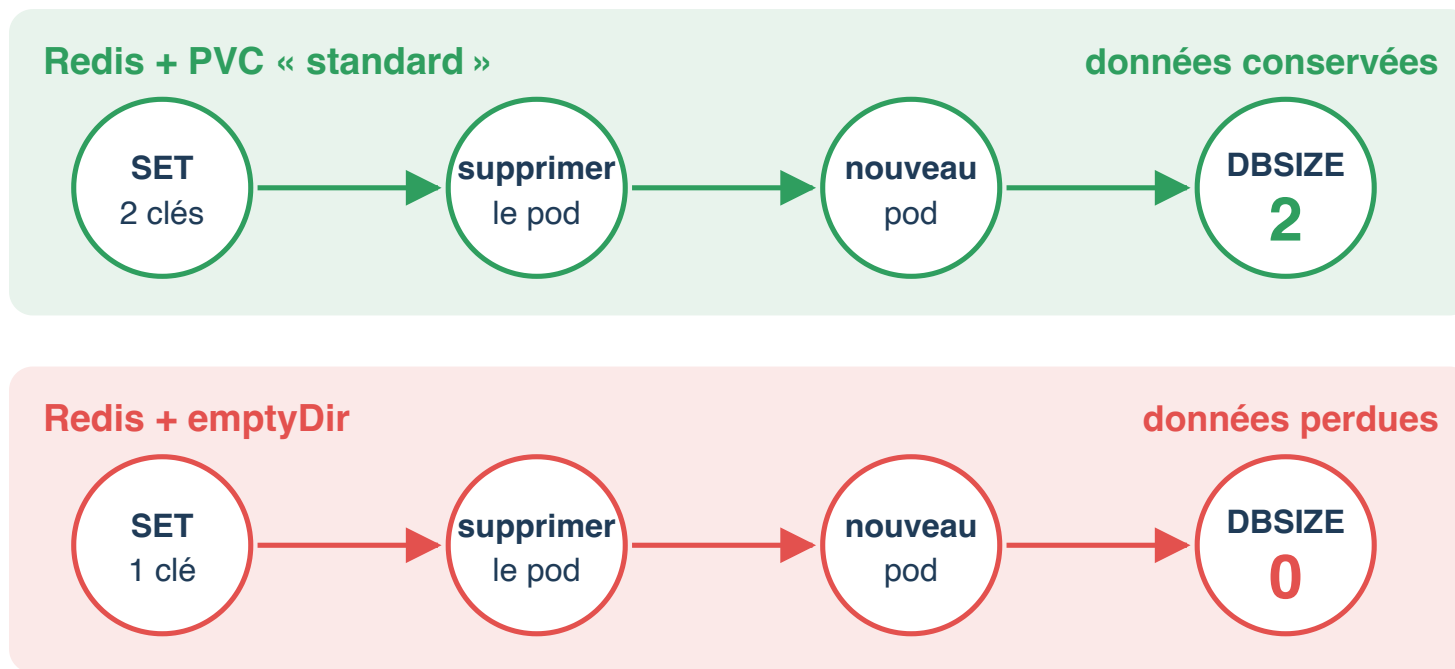
Provisionnement dynamique : on ne crée jamais le PV soi-même.

Modes d'accès et récupération

Mode d'accès	Abréviation	Usage
ReadWriteOnce	RWO	un seul nœud (disque bloc, local-path)
ReadWriteOncePod	RWOP	un seul pod dans tout le cluster
ReadOnlyMany	ROX	plusieurs nœuds en lecture
ReadWriteMany	RWX	plusieurs nœuds (NFS, CephFS, EFS)

- `reclaimPolicy` : **Delete** supprime le volume avec le PVC (cas de `standard`), **Retain** le conserve.
- `WaitForFirstConsumer` : volume créé sur le nœud du premier pod.

Mesure : le cache survit-il au pod ?

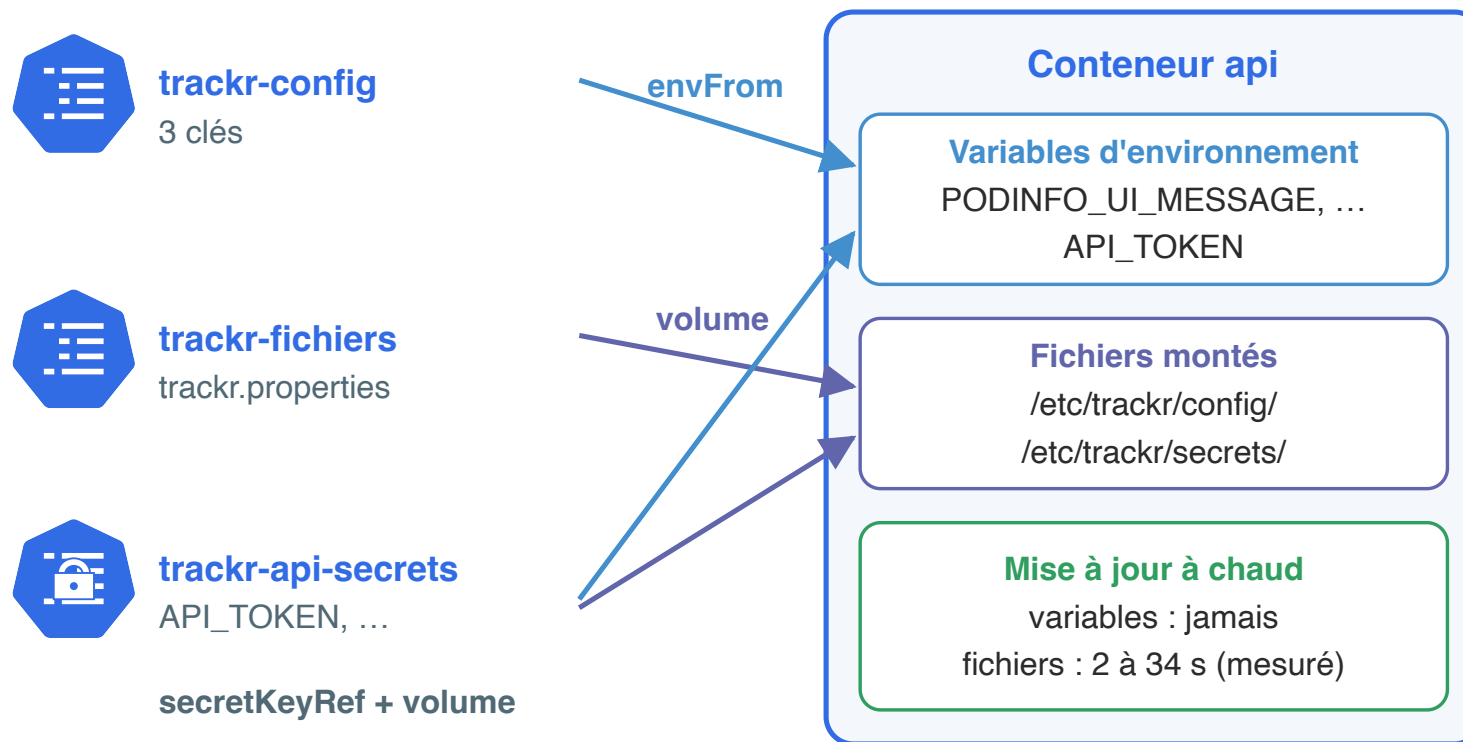


Mesuré sur le lab : DBSIZE avant / après suppression du pod

Redis avec `--appendonly yes`, pod supprimé puis recréé par le Deployment.

4. Configuration applicative

ConfigMap et Secret dans le conteneur



Un Secret n'est pas chiffré

```
$ kubectl get secret trackr-api-secrets \
  -o jsonpath='{.data.API_TOKEN}'
bGd2LWR1bW8tN2YzYTljMmU=
$ ... | base64 -d
lgv-demo-7f3a9c2e
```

Base64 est un **encodage**. Protégez les Secrets par RBAC et par le chiffrement au repos d'etcd (module 7).



env, envFrom et fichiers montés

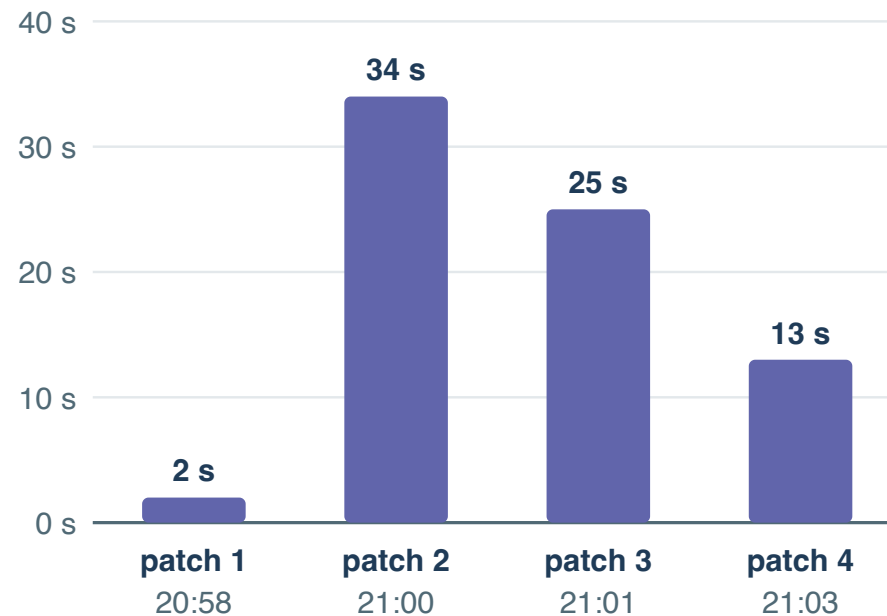
```
envFrom:  
- configMapRef:  
  name: trackr-config  
env:  
- name: API_TOKEN  
  valueFrom:  
    secretKeyRef:  
      name: trackr-api-secrets  
      key: API_TOKEN
```

```
volumes:  
- name: config  
  configMap:  
    name: trackr-fichiers  
- name: secrets  
  secret:  
    secretName: trackr-api-secrets  
    defaultMode: 0440
```

Pièges vus sur le lab : une clé `trackr.properties` devient une variable via `envFrom`, et un Secret en `0400` est illisible par l'utilisateur `app` sans `fsGroup`.

Mesure : la mise à jour à chaud

Délai avant mise à jour du fichier monté



Variable
d'environnement

jamais

mise à jour :
il faut recréer
le pod

Quatre `kubect1 patch configmap` : le fichier suit en 2 à 34 s, la variable jamais. Mesuré sur le lab.

5. Affecter les pods aux nœuds

Le scheduler : filtrer puis noter



Pending



1. Filtrage (éliminatoire)

nodeSelector, affinité required
taints non tolérés
requests > place libre
anti-affinité requise
spread DoNotSchedule

control-plane **taint non toléré**

worker **retenu**

worker2 **retenu**



2. Score (préférences)

affinités preferred
étalement des réplicas
équilibre des ressources

worker



worker2



3. Binding : le meilleur nœud

Labels de nœuds et nodeSelector

```
kubectl label node \
k8s-formation-worker2 disque=ssd
```

```
spec:
  nodeSelector:
    disque: ssd
```

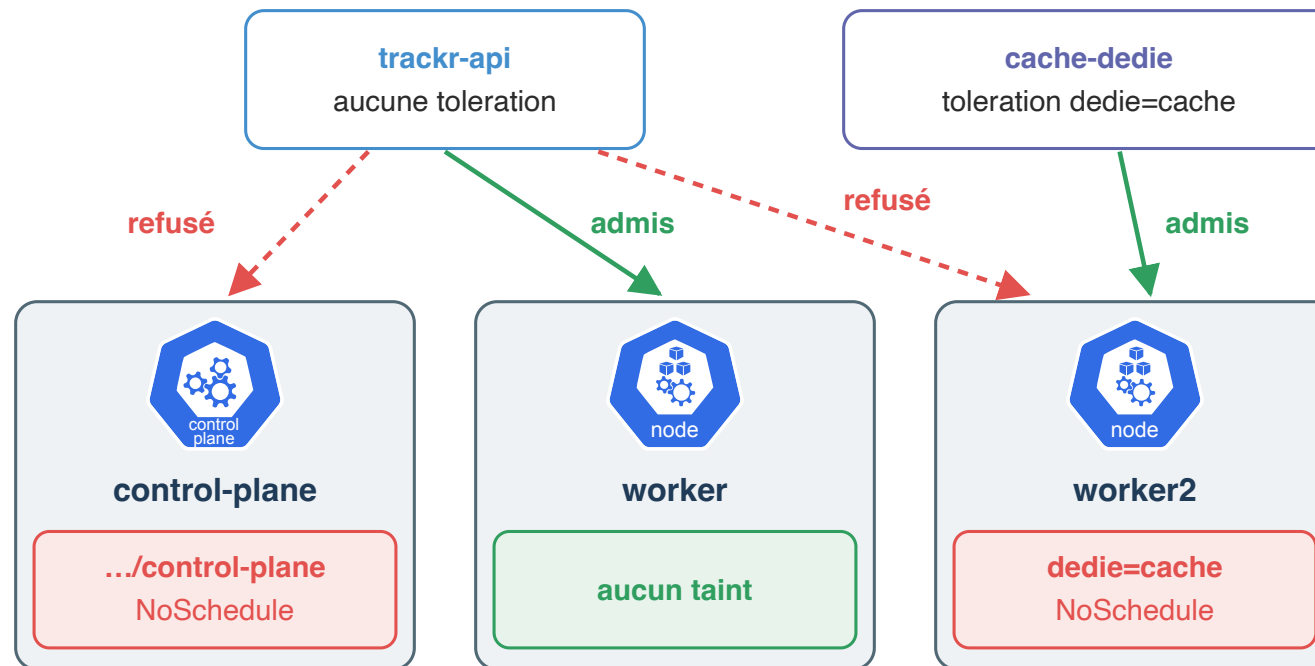
- Sans nœud étiqueté : **Pending**.
- Dès le label posé : planifié sur `worker2` en quelques secondes.
- Label retiré ensuite : le pod **reste** en place (IgnoredDuringExecution).

nodeAffinity : obligation et préférence

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
        - matchExpressions:
            - {key: node-role.kubernetes.io/control-plane, operator: DoesNotExist}
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 80
        preference:
          matchExpressions:
            - {key: disque, operator: In, values: ["ssd"]}
```

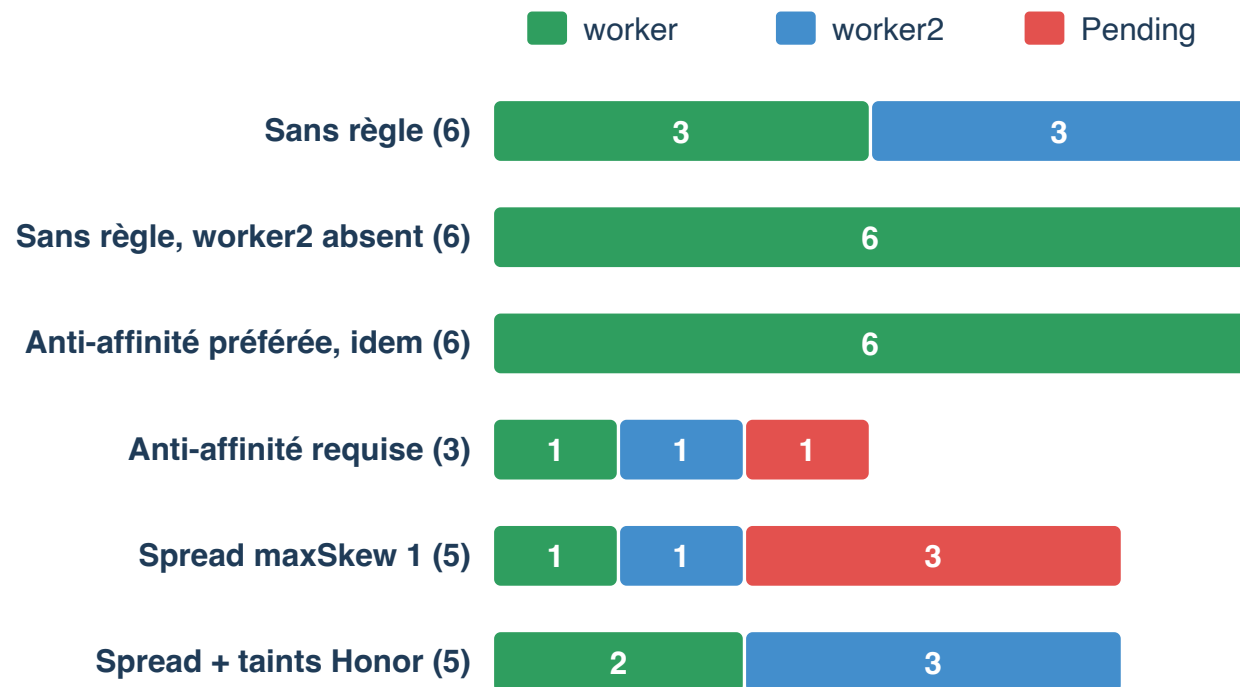
required filtre, preferred ajoute des points au score. Opérateurs : In, NotIn, Exists, DoesNotExist, Gt, Lt.

Taints et tolerations



Le taint **repousse**, la toleration **autorise**, le label **attire**. Effets : NoSchedule, PreferNoSchedule, NoExecute.

Mesure : répartir les réplicas



Nombre de réplicas par nœud, mesuré sur le lab (2 workers schedulables)

Rien n'est jamais rééquilibré après coup : les règles agissent au moment du placement.

topologySpreadConstraints sans piège

```
topologySpreadConstraints:  
  - maxSkew: 1  
    topologyKey: kubernetes.io/hostname  
    whenUnsatisfiable: DoNotSchedule  
    nodeTaintsPolicy: Honor  
    matchLabelKeys: ["pod-template-hash"]  
    labelSelector:  
      matchLabels:  
        app.kubernetes.io/name: trackr-front
```

- Sans `nodeTaintsPolicy: Honor`, le control-plane compte comme un nœud vide : 3 pods sur 5 Pending.
- Sans `matchLabelKeys`, 3 rollouts sur 6 ont mis les 2 fronts sur le même nœud.

Lire un FailedScheduling

`kubectl describe pod` ou `kubectl get events --field-selector reason=FailedScheduling` :

Fragment du message	Cause	Remède
« Insufficient memory »	requests trop grandes	réduire, ajouter des nœuds
« didn't match Pod's node affinity/selector »	aucun nœud étiqueté	poser le label
« had intolerated taint(s) »	taint non toléré	toleration
« didn't match pod anti-affinity rules »	trop de réplicas	« preferred »
« didn't match pod topology spread constraints »	maxSkew bloquant	« nodeTaintsPolicy »

« 0/3 nodes are available » : chaque nœud est compté dans une des causes.

Démo, TP et exercice

Démo 4 : OOMKilled en direct, throttling, Redis sur PVC, ConfigMap à chaud, taint et Pending.

`demos/demo-04-ressources-stockage-placement.md`

TP 4 (50 min) : configurer TrackR (ressources, ConfigMap et Secret, Redis persistant, init container, répartition).

`tp/tp-04-configurer-trackr.md`

Exercice 4 : prédire la classe de QoS et l'ordre d'éviction de six pods.

`exercices/exercice-04-qos.md`

À retenir

- Requests = placement, limits = plafond : CPU freiné, mémoire tuée.
- La QoS découle des ressources et décide de l'ordre d'éviction.
- Données durables : PVC sur une StorageClass, jamais emptyDir ni hostPath.
- Variables figées au démarrage, fichiers montés mis à jour.
- Placement : labels attirent, taints repoussent ; lisez FailedScheduling.

